Related Data with Flask

Session 5

Projects, tasks and shared tags

IN432 Advanced Web Technologies
Lebanese University
Master 1 Computer Science

The Persistence Lifecycle

Operation	Effect
Task(...)	Construct a Python object
db.session.add(task)	Track a new object
db.session.flush()	Send pending SQL inside the transaction
db.session.commit()	Flush and complete the transaction

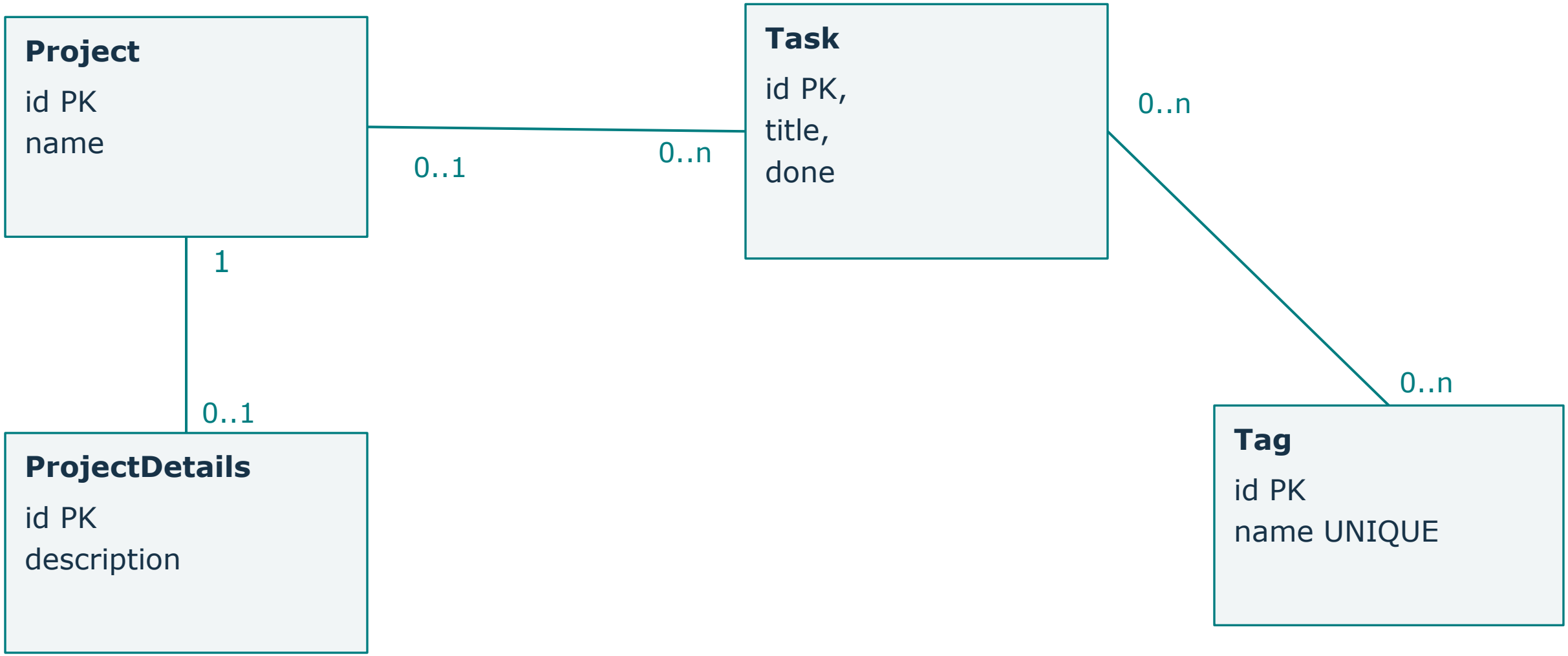
A task has an id after flush. Has it necessarily been committed?

Lina's Next Requirements

New requirement	Relationship
Group tasks into projects	One to many
Keep one optional details record per project	One to one
Share labels across several tasks	Many to many

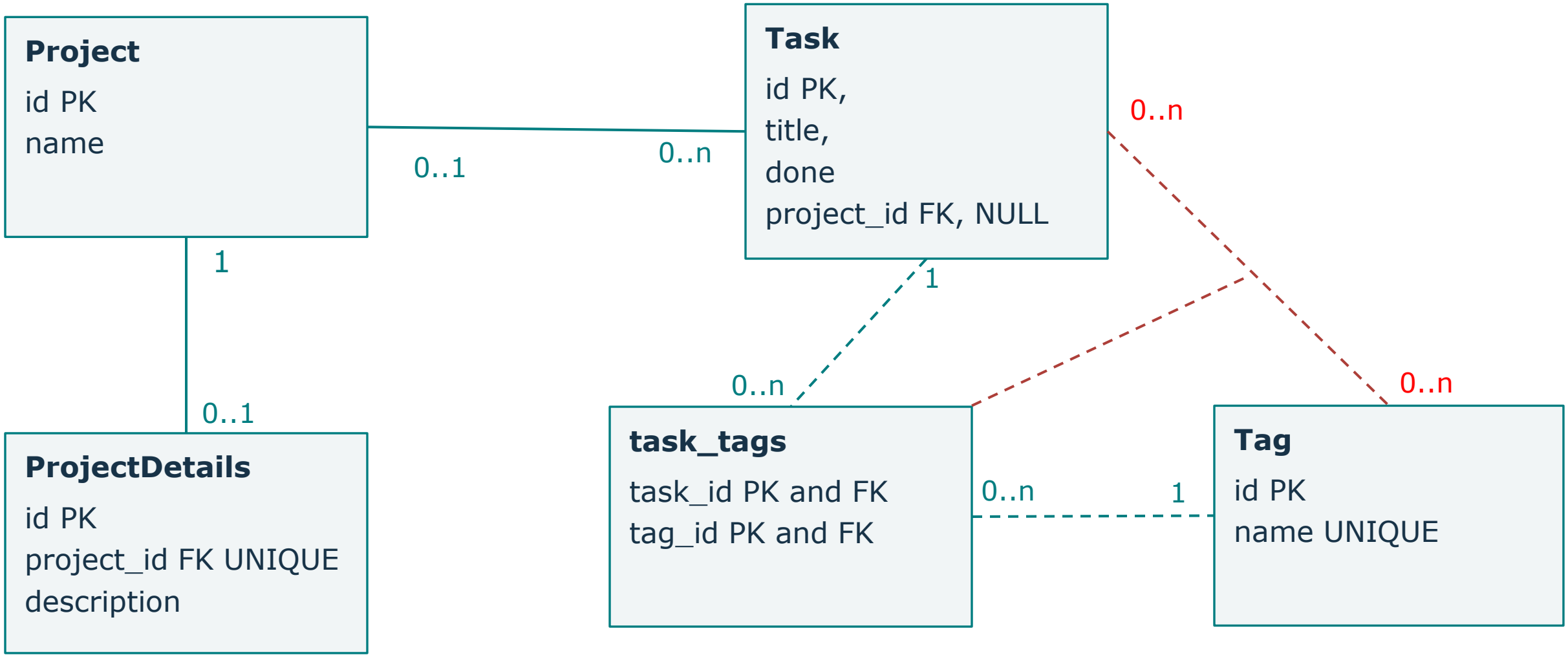
The same persistent Task API now needs related data.

Our Data Model



Four mapped classes and one association table

Our Data Model



Four mapped classes and one association table

One to Many Relationships

Example rows

projects

id	name
7	Coursework

tasks

id	title	project_id
31	Plan API	7
32	Review schema	7
33	Read docs	NULL

The many side stores the foreign key.

project.tasks is a collection. task.project is one object or None.

The Project and Task Mappings

task_api/models.py

```
# Inside Project
tasks = db.relationship(
    "Task", back_populates="project"
)
# Inside Task
project_id = db.Column(
    db.Integer,
    db.ForeignKey("projects.id"), nullable=True
)
project = db.relationship(
    "Project", back_populates="tasks"
)
```

- ForeignKey names a table and column.
- relationship names a mapped class.
- Each back_populates names the opposite attribute.

Paired Relationships with back_populates

Inside an application context with loaded objects

```
task.project = project
db.session.commit()

print(task.project.name)
print([item.id for item in project.tasks])
```

- The two attributes describe one relationship.
- Flush synchronizes the stored foreign key.
- Commit completes the transaction.

Project.tasks pairs with Task.project

Project Assignment in the Store

task_api/store.py

```
def assign(self, task_id, project_id):
    task = require(Task, task_id)
    project = None
    if project_id is not None:
        project = require(Project, project_id)
    task.project = project
    self._commit()
    return task.to_dict()
```

- require(...) loads by id or raises 404.
- Load the Project before changing Task.
- None removes the assignment.

JSON null becomes Python None.

Checkpoint Project Assignment

Example: task 31 belongs to project A with id 7.

Move it to project B with id 12

```
PUT /api/tasks/31/project
{"project_id": 12}
```

Which relationship changes? Which Task values should remain unchanged?

Try null and a missing id, then restore project 7.

One to One Relationships

- A Project may have one ProjectDetails object or none.
- Each details row belongs to one Project.
- A unique foreign key limits the number of matching rows.
- Scalar navigation gives access to one object.

Optionality and maximum cardinality are separate rules.

The ProjectDetails Mapping

task_api/models.py

```
# Inside ProjectDetails
project_id = db.Column(
    db.Integer,
    db.ForeignKey("projects.id"),
    nullable=False, unique=True
)
project = db.relationship(
    "Project", back_populates="details"
)
# Inside Project
details = db.relationship(
    "ProjectDetails",
    back_populates="project", uselist=False
)
```

- unique=True enforces at most one details row.
- uselist=False gives one object or None.
- A project can still have no details.

Creating and Updating Details

task_api/store.py

```
def put_details(self, project_id, fields):
    project = require(Project, project_id)
    created = project.details is None
    if created:
        project.details = ProjectDetails(
            description=fields["description"]
        )
    else:
        project.details.description = (
            fields["description"]
        )
    self._commit()
    return project.details.to_dict(), created
```

- First PUT creates the row and returns 201.
- Later PUT updates the same row and returns 200.
- The details id stays the same.

Checkpoint The Unique Foreign Key

Attempted details rows in one transaction

id	project_id	description
44	7	First details row
45	7	Second details row

Can both rows commit? Which declaration decides?

What must the application do after a failed commit?

Many to Many Relationships

Example rows

tasks

id	title
31	Plan API
32	Review schema

task_tags

task_id	tag_id
31	2
31	5
32	2

tags

id	name
2	database
5	urgent

Each association row stores one task and tag pair.

The Task Tag Association Table

task_api/models.py at module level

```
task_tags = db.Table(
    "task_tags",
    db.Column(
        "task_id", db.Integer,
        db.ForeignKey("tasks.id"),
        primary_key=True
    ),
    db.Column(
        "tag_id", db.Integer,
        db.ForeignKey("tags.id"),
        primary_key=True
    )
)
```

- Two foreign keys connect the tables.
- Both columns form the primary key.
- The same pair can appear only once.

Two Collections One Association Table

task_api/models.py

```
# Inside Task
tags = db.relationship(
    "Tag", secondary=task_tags,
    back_populates="tasks"
)

# Inside Tag
tasks = db.relationship(
    "Task", secondary=task_tags,
    back_populates="tags"
)
```

- secondary selects the association table.
- task.tags contains Tag objects.
- tag.tasks contains Task objects.

Attaching an Existing Tag

task_api/store.py

```
def attach_tag(self, task_id, tag_id):
    task = require(Task, task_id)
    tag = require(Tag, tag_id)
    if tag not in task.tags:
        task.tags.append(tag)
    self._commit()
    return task.to_dict()
```

- Reuse the existing Tag.
- Append a missing association.
- Commit writes the link.

Repeating this PUT keeps one association.

Detaching a Shared Tag

Remove tag 2 from task 31

Before

task_id	tag_id
31	2
31	5
32	2

After

task_id	tag_id
31	5
32	2

With the task, tag and existing association loaded

```
task.tags.remove(tag)
db.session.commit()
```

ORM Objects and Result Rows

Selection	Result extraction	Items you receive
select(Task)	scalars().all()	Task objects
select(Task.title, Project.name)	all()	Rows with both values

The selected values determine the result shape.

Reading Different Result Shapes

A mapped Task object

```
task = objects[0]
print(task.title)
```

Example output
Plan API

A row of selected values

```
row = rows[0]
print(row.title)
print(row.name)
```

Example output
Plan API
Coursework

Column names become Row attribute names.

What would scalars() keep from the two-column query?

Tasks with Their Project Names

task_api/store.py

```
statement = (
    db.select(
        Task.id,
        Task.title,
        Project.name.label("project_name")
    )
    .select_from(Task)
    .outerjoin(Task.project)
    .order_by(Task.id)
)
rows = db.session.execute(statement).all()
```

- The report starts from Task.
- The outer join keeps unassigned tasks.
- Their project_name becomes None.

What disappears if outerjoin becomes join?

Filtering Tasks by Tag

task_api/store.py

```
statement = (
    db.select(Task)
    .join(Task.tags)
    .where(Tag.id == tag_id)
    .order_by(Task.id)
)
tasks = db.session.execute(
    statement
).scalars().all()
```

- The join follows task_tags.
- The condition selects the requested tag.
- The selected entity is still Task.

GET /api/tasks?tag_id=2

Task Counts Including Empty Projects

task_api/store.py

```
statement = (
    db.select(
        Project.id, Project.name,
        func.count(Task.id).label("task_count")
    )
    .outerjoin(Project.tasks)
    .group_by(Project.id, Project.name)
    .order_by(Project.id)
)
rows = db.session.execute(statement).all()
```

Example report

Project	Count
Coursework	2
Future Ideas	0

Why does this report use COUNT(Task.id) rather than COUNT(*)?

The Public Task Representation

task_api/models.py

```
def to_dict(self):
    return {
        "id": self.id,
        "title": self.title,
        "done": self.done,
        "project_id": self.project_id,
        "tag_ids": [
            tag.id for tag in self.tags
        ]
    }
```

Example JSON response

```
{
  "id": 31,
  "title": "Plan API",
  "done": false,
  "project_id": 7,
  "tag_ids": [5]
}
```

Choose a boundary for related data in every response.

A Project Bundle as One Operation

POST /api/project-bundles request body

```
{
  "name": "Team Deliverable",
  "description": "Created as one unit",
  "tasks": [
    {"title": "Design the API"},
    {"title": "Implement the models"}
  ]
}
```

- Validate every child first.
- Store the related rows in one transaction.
- Use one final commit.

What happens if the project commits before a task insert fails?

Building the Project Bundle

task_api/store.py

```
def create_bundle(self, fields):
    try:
        project = Project(name=fields["name"])
        db.session.add(project)
        db.session.flush()
        project.details = ProjectDetails(
            description=fields["description"]
        )
        # Continue on the next slide.
```

- Register the parent.
- Flush inside the try block.
- Connect its details object.

Flush is still inside the uncommitted transaction.

One Commit and Rollback Boundary

task_api/store.py

```
        for item in fields["tasks"]:
            task = Task(
                title=item["title"],
                done=item.get("done", False)
            )
            task.project = project
            db.session.add(task)
        db.session.commit()
    except SQLAlchemyError:
        db.session.rollback()
        raise ApiError(
            500, "database_error",
            "Database operation failed"
        )
    return self.get(project.id)
```

- Continuation of the same try block.
- One commit follows all related writes.
- Rollback covers database failures, including flush.

Validation and Database Failures

Experiment	Boundary exercised
Invalid bundle sent through HTTP	Whole-request validation before the store
Constraint failure in the isolated store checker	Rollback after a flushed parent

Which experiment proves that a flushed Project can be rolled back?

After rollback, a later valid operation should succeed.

Relationship Deletion Policies

Operation	Chosen behavior in this application
Detach a tag	Remove one association and keep both entities
Delete a task	Remove its links and keep shared tags
Delete a project	Keep tasks and set their project to None
	Project details : Delete the details owned by the deleted project

Cardinality and deletion policy describe different rules.

Project Deletion with Task Preservation

task_api/store.py

```
def delete(self, project_id):
    project = require(Project, project_id)
    for task in list(project.tasks):
        task.project = None
    db.session.delete(project)
    self._commit()
```

- A copy supports safe iteration while detaching.
- Project.details uses cascade="all, delete-orphan".
- Commit coordinates the changes.

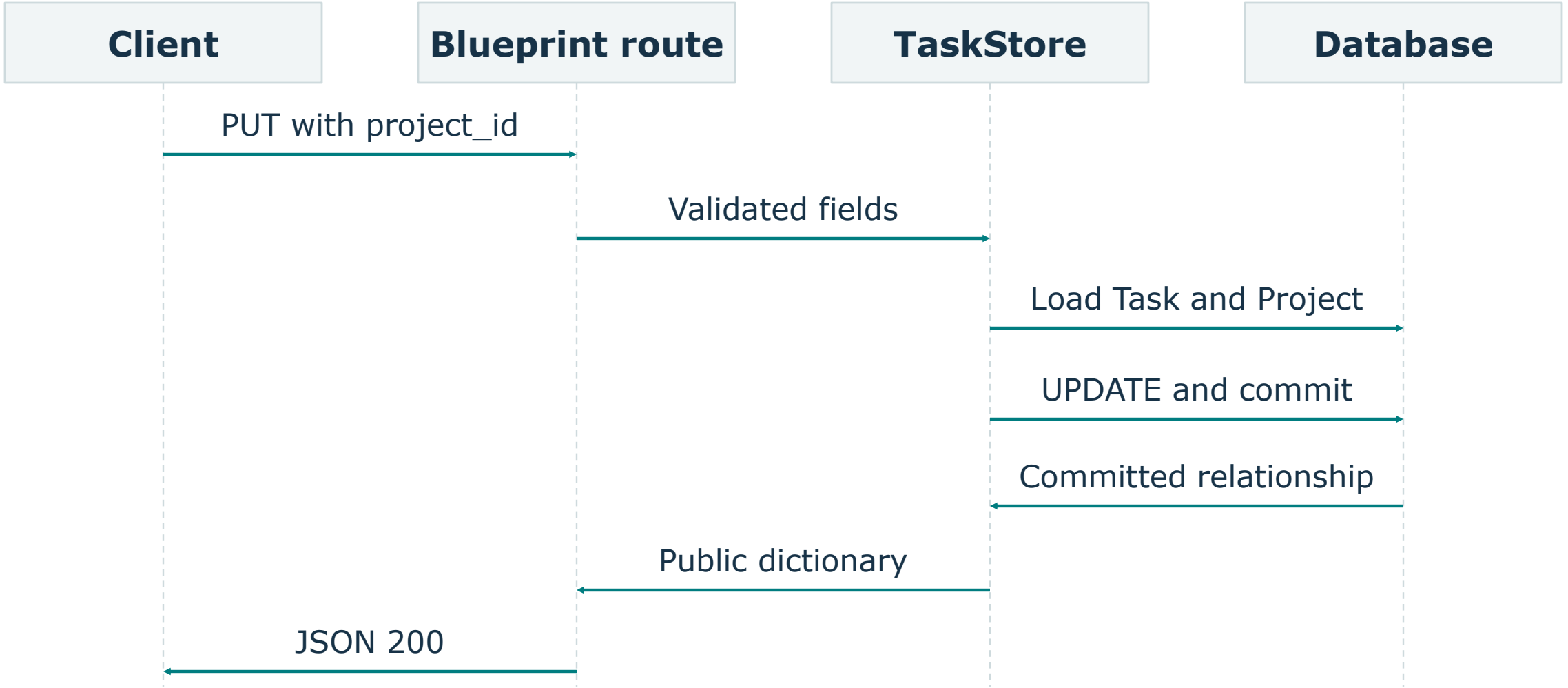
Shared Tag rows remain available.

Model Changes and Existing Tables

Situation	Action
Missing tables in a new database	create_all()
A new column in an existing table	Reviewed schema migration
This teaching lab	Separate instance/session5.db
Bring forward Session 4 tasks	Optional read-only export and separate import

Keep the original Session 4 database file.

An Assignment Request Across the Backend



Exit Questions

- Where is the foreign key stored in the Project Task relationship?
- Why does one to one need both scalar access and a database constraint?
- What changes when a tag is detached?
- Why does the empty-project report use COUNT(Task.id)?
- How does rollback affect a flushed project?
- What evidence shows that relationships survive a server restart?

Official References

Flask-SQLAlchemy Quickstart

Application Context

Basic Relationship Patterns

Session Basics

Cascades

ORM SELECT Statements

Relationship Loading

SQLite Foreign Key Support

Alembic Autogeneration